Challenge1

```
FSB.c = (~) - VIM
1 #include <stdio.h>
2
3 int main(int argc, char *argv[]){
4     printf(argv[1]);
5     return 0;
6 }
7
```

```
yun@yun-VirtualBox:~$ gcc FSB.c -o FSB
FSB.c: In function ‘main’:
FSB.c:4:9: warning: format not a string literal and no format arguments [-Wforma
t-security]
    4 |         printf(argv[1]);
      |         ^~~~~~
yun@yun-VirtualBox:~$ ./FSB
```

```
yun@yun-VirtualBox: ~
yun@yun-VirtualBox:~$ ./ FSB.c "%x %x %x %x"
bash: ./: 디렉터리입니다
yun@yun-VirtualBox:~$

yun@yun-VirtualBox:~$ ./FSB "%p %p %p %p"
0x7ffea4c5db88 0x7ffea4c5dba0 0x55643b828dc0 (nil)
```

```
0x7ffea4c5db88 0x7ffea4c5dba0 0x55643b828dc0 (nil)yun@yun-VirtualBox:~$ ./FSB "%
d %d %d %d"
-2053813032 -2053813008 2054958528 0yun@yun-VirtualBox:~$
```

```
(null)yun@yun-VirtualBox:~$ ./FSB "%d %d %d %d"
%d %d %d %dyun@yun-VirtualBox:~$
```

printf(argv[1]);의 경우, 포맷 지정자를 생략하고 사용자의 입력값 자체를 첫 번째 인자 자리에 넣은 상태이다. 이 경우, 첫 번째 인자로 들어온 문자열을 무조건 자신이 해석해야 할 명령어, 즉, 포맷이라고 맹신하는 상태가 된다. 따라서 %x라는 입력에 대해 이 뒤에 출력할 변

수 3개가 당연히 따라왔을 것이라는 전제 하여 레지스터와 스택 메모리를 탐색한다. 이로 인해 이전에 그 레지스터에 남아있던 다른 프로그램의 데이터나 주소값들을 출력하게 된다. 그에 반해 printf를 어떤 정보를 줄 것인지 미리 컴퓨터에게 명시하고, 그 형식에 맞는 자료를 요구하기에 첫 번째와는 다른 결과가 출력된다.

Challenge2

repeat.c (~) - VIM

```c
#include <stdio.h>

int main(){
    int sum = 0;

    sum = 0;
    for(int i = 1; i <= 10; i++){
        sum += i;
    }
    printf("for sum: %d\n", sum);


    sum = 0;
    int j = 1;
    while (j <= 10){
        sum += j;
        j++;
    }
    printf("while sum: %d\n", sum);


    sum = 0;
    int k = 1;
     sum = 0;
     int k = 1;
     do {
         sum += k;
         k++;
     } while(k <= 10);
     printf("do-while-sum: %d\n", sum);

     return 0;
}
```

```
yun@yun-VirtualBox:~$ ./repeat
for sum: 55
while sum: 55
do-while-sum: 55
```

- for
초기식, 조건식, 증감식이 () 안에 한 줄로 모두 모여 있음.
반복의 시작과 끝, 변화량을 한눈에 파악하기에 가장 좋음
반복 횟수가 명확하게 정해져 있을 때 사용하기 유용

-while
() 안에 오직 조건식만 들어감.
초기값은 반복문 진입 전에 미리 설정해야 함
증감식은 반복문 블록 내부에서 직접 처리
몇 번 반복할지는 모르겠지만, 특정 조건이 유지되는 동안 실행할 때 유용

-do-while
{}블록이 먼저 오고, 조건 검사 while()이 맨 마지막에 붙음
조건의 참 거짓 여부와 상관없이 무조건 코드를 최소 1번은 실행해야할 때 유용

challenge3

cond.c (~) - VIM

```c
#include <stdio.h>

int main(){
    int num;
    printf("정수를 입력하세요: ");
    scanf("%d", &num);

    if (num > 0) printf("if: 양수입니다.\n");
    else if (num <0) printf("if: 음수입니다.\n");
    else printf("if: 0입니다.\n");

    printf("삼항: %s\n", (num > 0) ? "양수입니다." : ((num<0) ? "음수입니다."  : "0입니다."));

    int state = (num > 0) ? 1 : ((num< 0) ? -1 : 0);
    switch (state){
        case 1 : printf("switch: 양수입니다.\n"); break;
        case -1 : printf("switch: 음수입니다.\n"); break;
        case 0 : printf("switch: 0입니다. \n"); break;
        }
    return 0;
}
```

"cond.c" 21L, 597B 1,1 모두

```
yun@yun-VirtualBox:~$ gcc cond.c -o cond
yun@yun-VirtualBox:~$ ./cond 20
정수를 입력하세요: 20
if: 양수입니다.
삼항: 양수입니다.
switch: 양수입니다.
```

challenge4

```
(gdb) break main
Breakpoint 1 at 0x11b5: file cond.c, line 3.
(gdb) r
Starting program: /home/yun/cond
```

```
(gdb) r
Starting program: /home/yun/cond

This GDB supports auto-downloading debuginfo from the following URLs:
  <https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at cond.c:3
3       int main(){
(gdb) n
5               printf("정수를 입력하세요: ");
(gdb) n
6               scanf("%d", &num);
```

```
(gdb) s
__isoc99_scanf (format=0x555555556020 "%d") at ./stdio-common/isoc99_scanf.c:29
warning: 29     ./stdio-common/isoc99_scanf.c: 그런 파일이나 디렉터리가 없습니다
(gdb) s
30      in ./stdio-common/isoc99_scanf.c
(gdb) s
__vfscanf_internal (s=0x7ffff7e038e0 <_IO_2_1_stdin_>,
    format=0x555555556020 "%d", argptr=argptr@entry=0x7fffffffdcd0,
    mode_flags=mode_flags@entry=2) at ./stdio-common/vfscanf-internal.c:278
warning: 278    ./stdio-common/vfscanf-internal.c: 그런 파일이나 디렉터리가 없습
니다
```

```
(gdb) i r
rax            0x5555555551a9      93824992235945
rbx            0x7fffffffdee8      140737488346856
rcx            0x555555557da8      93824992247208
rdx            0x7fffffffdef8      140737488346872
rsi            0x7fffffffdee8      140737488346856
rdi            0x1                 1
rbp            0x7fffffffddc0      0x7fffffffddc0
rsp            0x7fffffffddb0      0x7fffffffddb0
r8             0x0                 0
r9             0x7ffff7fca380      140737353917312
r10            0x7fffffffdae0      140737488345824
r11            0x203               515
r12            0x1                 1
r13            0x0                 0
r14            0x555555557da8      93824992247208
r15            0x7ffff7ffd000      140737354125312
rip            0x5555555551b5      0x5555555551b5 <main+12>
eflags         0x202               [ IF ]
cs             0x33                51
ss             0x2b                43
ds             0x0                 0
(gdb) x/10wx $rsp
0x7fffffffddb0: 0xffffdea0      0x00007fff      0xffffdee8      0x00007fff
0x7fffffffddc0: 0xffffde60      0x00007fff      0xf7c2a1ca      0x00007fff
0x7fffffffddd0: 0xffffde10      0x00007fff
```